

Algoritmy a grafy 1 (BI-AG1), Přednáška č. 12

Hledání nejkratších cest v ohodnoceném grafu

Dušan Knop, Michal Opler, Tomáš Valla

Fakulta informačních technologií
České vysoké učení technické v Praze
<https://courses.fit.cvut.cz/BI-AG1/>



ZS 2025/2026, 15. prosince 2025

(Verze dokumentu: 8. 1. 2026 12:57)

Ohodnocený orientovaný graf

- Problém minimální kostry z minulé přednášky byl řešen nad souvislým neorientovaným grafem ohodnoceným pomocí **váhové funkce** w , $w(e)$ byla váha hrany e .
- Vstup pro problém nejkratších cest je podobný. Ohodnocení hrany interpretujeme jako její **délku**, značíme $\ell(e)$.
- Délku hrany můžeme přirozeně rozšířit i na cesty: délka cesty je součet délek jejích hran.
- Uvažujme **zatím** pouze **kladná ohodnocení**: $\forall e : \ell(e) > 0$.
- Dále budeme uvažovat orientované grafy $G = (V, E)$. Nicméně vše, co v této přednášce vysvětlíme, platí i pro neorientované grafy.

Vzdálenost v ohodnoceném grafu

Definice

Pro libovolné dva vrcholy u a v ohodnoceného orientovaného grafu, **vzdálenost** $d(u, v)$ je minimum z délek všech uv -cest (cest z u do v), případně $+\infty$, pokud žádná uv -cesta neexistuje.

- Toto minimum je vždy dobře definované, protože pro libovolné $u, v \in V$ existuje v grafu pouze konečně mnoho uv -cest.
- Pozor na to, že v orientovaném grafu se $d(u, v)$ a $d(v, u)$ mohou lišit.

Definice

Nejkratší uv -cesta je libovolná uv -cesta, jejíž délka je rovná vzdálenosti $d(u, v)$.

Připomenutí:

Definice

Posloupnost $(v_0, e_1, v_1, \dots, e_k, v_k)$ v orientovaném grafu $G = (V, E)$ se nazývá v_0v_k -**sled** (nebo krátce **sled**), pokud

- $v_i \in V$ pro všechna $i \in \{0, \dots, k\}$ a
- $e_i = (v_{i-1}, v_i) \in E$ pro všechna $i \in \{1, \dots, k\}$.

Délka $\ell(S)$ sledu S je součet délek jeho hran.

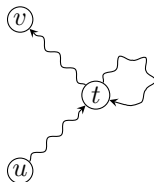
Zkracování sledů

Lemma (o zjednodušování sledů)

Pokud jsou délky hran kladné, pak pro každý uv -sled existuje uv -cesta stejné nebo menší délky.

Důkaz.

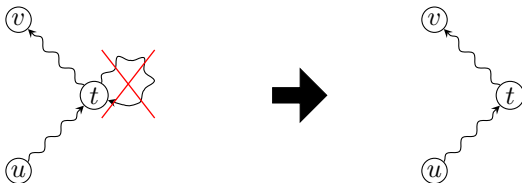
- Pokud sled není cestou, znamená to, že se v něm opakují vrcholy.
- Označme tedy t první vrchol (v pořadí od u), který se opakuje.
- ...



Zkracování sledů

Dokončení důkazu.

- Část sledu mezi prvním a posledním výskytem vrcholu t vystříháme a získáme uv -sled stejné nebo menší délky, protože délky hran jsou kladné.
- Přitom ubyla alespoň jedna hrana, takže opakováním tohoto postupu časem dostaneme cestu. □



Nejkratší sled

Důsledek

Pokud jsou délky hran v $G = (V, E)$ kladné, pak pro libovolné dva vrcholy $u, v \in V$ buď neexistuje žádný uv -sled nebo existuje nejkratší uv -sled a ten je současně nejkratší uv -cestou.

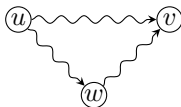
Důkaz.

- Nejkratší cesta je jedním ze sledů.
- Pokud by tvrzení neplatilo, musel by existovat nějaký kratší sled.
- Ten by ovšem, podle předchozího lemmatu, šel zjednodušit na ještě kratší cestu. □

Trojúhelníková nerovnost

Důsledek

Jsou-li délky hran kladné, platí pro vzdálenosti **trojúhelníková nerovnost**: $d(u, v) \leq d(u, w) + d(w, v)$ pro libovolné $u, v, w \in V$.



Důkaz.

- Pokud je $d(u, w)$ nebo $d(w, v)$ nekonečná, nerovnost triviálně platí.
- V opačném případě uvažme spojení nejkratší uw -cesty s nejkratší wv -cestou.
- To je nějaký uv -sled délky $d(u, w) + d(w, v)$ a ten, podle předchozího důsledku, nemůže být kratší než nejkratší uv -cesta, která má délku $d(u, v)$. □

Vrcholy na nejkratší cestě

Důsledek

Nechť jsou délky hran kladné, P je nejkratší uv -cesta a w leží na P . Označme P_{uw} část P mezi u a w a P_{wv} část P mezi w a v . Pak P_{uw} je nejkratší uw -cesta a P_{wv} je nejkratší wv -cesta.

Důkaz.

- Podle definice a předchozího důsledku platí
$$\ell(P) = d(u, v) \leq d(u, w) + d(w, v) \leq \ell(P_{uw}) + \ell(P_{wv}) = \ell(P).$$
- Tedy všude musí být rovnost a $\ell(P_{uw}) = d(u, w)$ a $\ell(P_{wv}) = d(w, v)$. □

Hledání nejkratších cest

- Jak najít nejkratší cestu mezi dvěma vrcholy ohodnoceného orientovaného grafu?

Fakt (bez důkazu)

Problém nalezení délky nejkratší cesty mezi dvěma vrcholy pro graf ohodnocený **obecnými** (tedy i zápornými) délkami je NP-těžký – proto pro něj neočekáváme existenci efektivního algoritmu.

- Pro některá speciální ohodnocení grafu efektivní algoritmy existují. V této přednášce si ukážeme dva nejpoužívanější:
 - ▶ **Dijkstrův**, který předpokládá **kladné délky** hran (případně nezáporné).
 - ▶ **Bellmanův-Fordův**, který připouští záporné délky, ale předpokládá **neexistenci záporných cyklů** v grafu.

Dijkstrův algoritmus pro hledání nejkratších cest

- Tento dnes asi nepoužívanější algoritmus pro hledání nejkratších cest vymyslel v roce 1959 prof. Dijkstra.
- **Dijkstrův algoritmus** (dále jen **Dijkstra**) řeší obecnější problém:
 - Vstup:** Orientovaný graf $G = (V, E)$ s kladnými délkami hran $\ell: E \rightarrow \mathbb{R}^+$ a počáteční vrchol v_0 .
 - Výstup:** Vzdálenosti z vrcholu v_0 do všech vrcholů grafu G .
- **Dijkstra** je zobecněním průchodu grafu do šířky.

Úprava BFS pro přirozené váhy

- Mějme nejdříve orientovaný graf, jehož hrany mají **přirozené délky**.
- Každou hranu nyní „podrozdělíme“ – nahradíme ji cestou tvořenou tolika jednotkovými hranami, kolik činila délka hrany.



- Tím vznikne neohodnocený graf, ve kterém můžeme nejkratší cesty nalézt **prohledáváním do šířky** – viz algoritmus [BFS](#).
- Tento algoritmus je funkční, ale neefektivní:
 - ▶ Označíme-li L maximální délku hrany, podrozdělením vznikne $O(L \cdot m)$ nových vrcholů a hran.
 - ▶ Složitost [BFS](#) algoritmu tedy bude $\Theta(L \cdot m + n)$.

Základní idea Dijkstra

- Místo simulace průchodu **BFS** vlny v lineárním čase vzhledem k délce hran, **Dijkstra** zpracuje každou hranu v $O(1)$ čase.
- Představme si, že v každém **původním** vrcholu je „budík“.
- Místo krokové simulace vyslání **BFS** vlny z vrcholu nastavíme budíky následníků na časy, kdy by do nich vlna dorazila (předpokládáme jednotkovou rychlost šíření vlny).
- Zjistíme, ve kterém sousedním vrcholu zazvoní první budík.
- Podíváme se, jaké hrany z tohoto vrcholu vedou dále, a spočítáme, kdy po nich vlna dorazí do dalších následníků.
- Pokud by do nějakého následníka dorazila dříve, než je nastaven jeho budík, nastavíme tento budík na tento lepší čas.
- Opět počkáme, v kterém vrcholu zazvoní první budík, a vše opakujeme pro něj.
- Toto opakujeme, dokud nezazvoní budíky ve všech vrcholech.

Základní idea Dijkstra

- Čas nastavený na budíku vrcholu v označíme $h(v)$.
- Počáteční hodnota nenastaveného budíku je $h(v) = +\infty$.
- Podobně jako při **BFS** průchodu budeme rozlišovat tři stavy vrcholů:
 - ▶ **nenalezené**,
 - ▶ **otevřené** (mají nastavené budíky),
 - ▶ a **uzavřené** (jejich budíky už zazvonily).
- Pro každý vrchol v vyjma počátečního v_0 si budeme pamatovat jeho **předchůdce** $P(v)$, abychom mohli nejkratší cesty zrekonstruovat.

Algoritmus **Dijkstra**($G, \ell: E \rightarrow \mathbb{R}^+, v_0$)

(1) Pro všechny vrcholy v :

(2) stav(v) := nenalezený

(3) $h(v) := +\infty$; $P(v) := \perp$

(4) stav(v_0) := otevřený

(5) $h(v_0) := 0$

(6) Dokud existují nějaké otevřené vrcholy:

(7) Vyber otevřený vrchol v , jehož $h(v)$ je nejmenší.

(8) Pro všechny následníky w vrcholu v : //relaxace v

(9) Pokud $h(w) > h(v) + \ell((v, w))$:

(10) $h(w) := h(v) + \ell((v, w))$

(11) stav(w) := otevřený

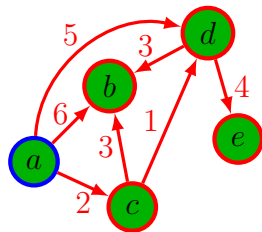
(12) $P(w) := v$

(13) stav(v) := uzavřený

(14) Vrať pole vzdáleností h a pole předchůdců P

Algoritmus **Dijkstra**($G, \ell: E \rightarrow \mathbb{R}^+, v_0$)

- (1) Pro všechny vrcholy v :
- (2) stav(v) := nenalezený
- (3) $h(v) := +\infty$; $P(v) := \perp$
- (4) stav(v_0) := otevřený
- (5) $h(v_0) := 0$
- (6) Dokud existují nějaké otevřené vrcholy:
- (7) Vyber otevřený vrchol v ,
 jehož $h(v)$ je nejmenší.
- (8) Pro všechny následníky w vrcholu v :
- (9) Pokud $h(w) > h(v) + \ell((v, w))$:
- (10) $h(w) := h(v) + \ell((v, w))$
- (11) stav(w) := otevřený
- (12) $P(w) := v$
- (13) stav(v) := uzavřený
- (14) Vrať pole vzdáleností h a
 pole předchůdců P



	a	b	c	d	e
$h :$	0	$+\infty$	$+\infty$	$+\infty$	$+\infty$
$P :$	$\perp$	$\perp$	a	a	c
	v	w	w	w	w

Relaxace vrcholů

Definice

Přepočítání ohodnocení $h(w)$ pro všechny následníky w vrcholu v v **Dijkstra** na řádcích (8)–(13) budeme nazývat **relaxace vrcholu** v .

- (8) Pro všechny následníky w vrcholu v : `//relaxace v`
- (9) Pokud $h(w) > h(v) + \ell((v, w))$:
- (10) $h(w) := h(v) + \ell((v, w))$
- (11) $\text{stav}(w) := \text{otevřený}$
- (12) $P(w) := v$
- (13) $\text{stav}(v) := \text{uzavřený}$

Konečnost a správnost Dijkstra

Věta

Dijkstra na grafu s kladnými délkami hran se v konečném čase zastaví a po jeho skončení budou všechny dosažitelné vrcholy v uzavřeny s $h(v) = d(v_0, v)$.

Tuto větu dokážeme kombinací 4 vlastností **Dijkstra**

Vlastnost O: Ohodnocení $h(v)$ v průběhu **Dijkstra** nikdy neroste a je-li konečné, rovná se délce nějakého sledu z v_0 do v .

Vlastnost M: V každém kroku výpočtu platí, že $h(z) \leq h(o)$ pro z uzavřený vrchol a o otevřený vrchol. Speciálně pak platí, že **Dijkstra** nikdy neotevře již uzavřený vrchol.

Vlastnost D: Když se **Dijkstra** zastaví, uzavřené jsou právě vrcholy dosažitelné z v_0 .

Vlastnost V: Když se **Dijkstra** zastaví, je $h(w) = d(v_0, w)$ pro všechny $w \in V$.

Vlastnost O (ohodnocení)

Ohodnocení $h(v)$ v průběhu **Dijkstra** nikdy neroste a je-li konečné, rovná se délce nějakého sledu z v_0 do v .

Důkaz.

- Indukcí podle doby běhu **Dijkstra**.
- Na počátku výpočtu tvrzení určitě platí, protože jediné konečné ohodnocení je $h(v_0) = 0$.
- Kdykoliv **Dijkstra** snižuje ohodnocení $h(w)$, stane se tak relaxací otevřeného vrcholu v s konečným $h(v)$, jehož následníkem je w .
- Podle indukčního předpokladu tedy existuje v_0v -sled délky $h(v)$.
- Jeho rozšířením o hranu vw vznikne v_0w -sled délky $h(v) + \ell((v, w))$, což je přesně nová konečná hodnota $h(w)$. $\square$

Vlastnost M (monotonie)

V každém kroku výpočtu platí, že $h(z) \leq h(o)$ pro z uzavřený vrchol a o otevřený vrchol. Speciálně pak platí, že **Dijkstra** nikdy neotevře již uzavřený vrchol.

Důkaz.

- Důkaz provedeme matematickou indukcí podle počtu relaxací.
- ZI: pokud ještě neproběhla žádná relaxace, neexistuje žádný uzavřený vrchol, jediný otevřený je v_0 a tvrzení triviálně platí.
- Indukční krok:
 - ▶ Algoritmus zvolí otevřený vrchol v s nejmenším $h(v)$.
 - ▶ Dle IP tedy před relaxací v platí $h(z) \leq h(v) \leq h(o)$ pro libovolný z uzavřený a o otevřený.
 - ▶ Nyní vrchol v relaxujeme: ...

Vlastnost M (monotonie)

V každém kroku výpočtu platí, že $h(z) \leq h(o)$ pro z uzavřený vrchol a o otevřený vrchol. Speciálně pak platí, že **Dijkstra** nikdy neotevře již uzavřený vrchol.

Důkaz.

- Indukční krok:

- ▶ Algoritmus zvolí otevřený vrchol v s nejmenším $h(v)$.
- ▶ Dle IP tedy před relaxací v platí $h(z) \leq h(v) \leq h(o)$ pro libovolný z uzavřený a o otevřený.
- ▶ Nyní vrchol v relaxujeme:
 - ▶ Pokud w byl uzavřený, nemůže se jeho hodnota změnit, neboť již před relaxací platilo $h(w) \leq h(v)$.
 - ▶ Pokud w byl otevřený nebo nenalezený, jeho hodnota se sice může snížit na $h(v) + \ell(v, w)$, ale nikdy ne pod $h(v)$, takže ani pod $h(z)$ žádného uzavřeného z .

- Kdybychom otevřeli již uzavřený vrchol z , muselo by platit $h(v) + \ell(v, z) \leq h(z)$, což nelze.



Vlastnost D (dosažitelnost)

Když se **Dijkstra** zastaví, uzavřené jsou právě vrcholy dosažitelné z v_0 .

Důkaz.

- Dokážeme stejně jako obdobnou vlastnost **BFS**.
- První otevřeným a následně uzavřeným vrcholem je v_0 .
- Vrchol je otevřen, právě když je do té doby nenalezeným následníkem dříve otevřeného vrcholu a tudíž je dosažitelný z v_0 .
- Protože **Dijkstra** dle Vlastnosti M, nikdy neotevře již uzavřený vrchol, zbytek důkazu je totožný s **BFS**:
- Kdyby tedy existoval nějaký dosažitelný, ale neuzavřený vrchol, existoval by ten „nejbližší“ (co do počtu hran na nejkratší cestě) a to by vedlo ke sporu. □

Vlastnost V (vzdálenost)

Když se **Dijkstra** zastaví, je $h(w) = d(v_0, w)$ pro všechny $w \in V$.

Důkaz.

- Inspirujeme se důkazem obdobné vlastnosti **BFS**.
- Vrchol w dosažitelný z v_0 nazveme **špatný**, pokud $h(w) \neq d(v_0, w)$.
- Z Vlastnosti O víme, že $h(w)$ odpovídá délce nějakého v_0w -sledu a tedy $h(w) > d(v_0, w)$.
- Obdobně jako pro **BFS** ukážeme sporem, že nemůže existovat žádný špatný vrchol, čímž bude důkaz dokončen.
 - ▶ Myšlenka je opět stejná – budeme předpokládat, že nějaké špatné vrcholy existují.
 - ▶ Vybereme si mezi nimi ten „nejbližší“ k v_0 a ukážeme, že nemohl být špatný.
 - ▶ Kdyby ale nějaký špatný byl, musel by být i „nejbližší“.

Vlastnost V

Pokud se **Dijkstra** zastaví, je $h(w) = d(v_0, w)$ pro všechny $w \in V$.

Pro spor předpokládejme, že existují špatné vrcholy.

- Víme, že v_0 není špatný, protože $h(v_0) = 0 = d(v_0, v_0)$.
- Buď w špatný vrchol takový, že nejkratší cesta v G z v_0 do w používá nejmenší možný počet hran.
- Buď v předchůdce w na této cestě z v_0 .
- Kdyby v byl špatný, volili bychom v namísto w . Tedy platí $h(v) = d(v_0, v)$.
- Vrchol v byl jistě někdy otevřený a později relaxovaný. Při jeho relaxaci byla aktuální hodnota $h(w)$ větší než $d(v_0, w)$ (neboť toto platí dokonce až po zastavení **Dijkstra**).
- V tom případě ale po relaxaci v muselo platit $h(w) = d(v_0, v) + \ell((v, w)) = d(v_0, w)$. Spor.



Vlastnost sledů v Dijkstra

Lemma

Je-li po uzavření vrcholu v v algoritmu **Dijkstra** $h(v)$ konečné pro nějaký vrchol v , rovná se délce nejkratší v_0v -cesty, která jako vnitřní vrcholy používá pouze uzavřené vrcholy.

Důkaz.

- Indukcí podle k počtu uzavřených vrcholů.
- Pro $k = 0$ platí, neboť jediné konečné je $h(v_0) = 0$.
- Pro indukční krok, kdy relaxujeme a tedy uzavíráme v , uvážíme dva případy:
 - ▶ Relaxace v nezměnila $h(w)$. Pak z IP tvrzení platí.
 - ▶ Relaxace v změnila $h(w)$. Pak dle Vlastnosti O $h(w)$ odpovídá délce nejkratší v_0v -cesty protažené o hranu (v, w) .
 - ▶ Protože nyní ale je v uzavřený a v_0v -cesta (dle IP) používala jako vnitřní vrcholy jen uzavřené vrcholy, platí tato vlastnost i pro w .



Složitost Dijkstra

Věta

Pokud jsou délky hran kladné, potom **Dijkstra** nad souvislým grafem o n vrcholech běží v čase $O(n^2)$.

Důkaz.

- Inicializace trvá $O(n)$.
 - Každý vrchol uzavřeme nejvýše jednou.
 - Vnější cyklem projdeme nejvýše n -krát.
 - Pokaždé hledáme minimum až z n ohodnocení vrcholů a procházíme až n následníků. □
-
- V grafech s malým počtem hran je odhad $O(n^2)$ zbytečně nadhodnocený.
 - Celkový počet operací na řádku (10) je pouze $O(m)$.
 - Ale brzdí nás hledání minima: $O(n)$.

Dijkstra s binární haldou

- Podobně jako u Jarníkova algoritmu, použijeme (binární) haldu, klíčem v bude hodnocení $h(v)$.

Algoritmus `DijkstraHalda`($G, \ell: E \rightarrow \mathbb{R}^+, v_0$)

(1) Pro všechny $v \in V$:

(2) $h(v) := +\infty$; $P(v) := \perp$

(3) $h(v_0) := 0$

(4) $H := \text{HeapBuild}(V)$ uspořádanou podle klíčů h

(5) Dokud H neprázdná:

(6) $v := \text{HeapExtractMin}(H)$

(7) Pro všechny následníky w vrcholu v :

(8) Pokud $w \in H$ & $h(w) > h(v) + \ell((v, w))$:

(9) $P(w) := v$

(10) $\text{HeapDecreaseKey}(H, w, h(v) + \ell((v, w)))$

(11) Vrať P a h

Složitost Dijkstra s binární haldou

Věta (o časové složitosti Dijkstra s binární haldou)

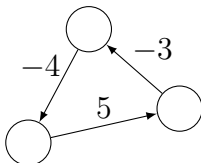
Časová složitost `DijkstraHalda` je $O(|E| \log |V|)$.

Důkaz.

- Řádky (1)-(4) trvají $O(|V|)$.
- Celková složitost na řádku (6) je $O(|V| \log |V|)$.
- Celková složitost na řádku (10) je $O(|E| \log |V|)$, neboť počet volání operace `HeapDecreaseKey` je nejvýše $|E|$. □

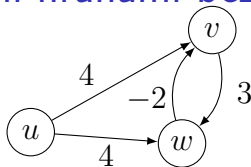
Hrany s nulovými či zápornými délkami

- Rozmysleme, co by se stalo, kdybychom měli graf s hranami záporné délky.



- Když v grafu existuje záporný cyklus, nejkratší sled není definován, protože ke každému sledu existuje ještě kratší.

Grafy se zápornými hranami bez záporných cyklů



Obrázek: Trojúhelníková nerovnost zůstává v platnosti

- Kdyby v grafu neexistoval záporný cyklus, sama existence hran záporné délky by problémy nepůsobila.
- Lemma o zjednodušování sledů a jeho důsledky by stále platily.
- Proto nyní připustíme grafy s hranami záporných délek, ale bez záporných cyklů.

Fakt (bez důkazu)

Dijkstra na grafech s hranami záporných délek bez záporných cyklů může vrcholy otevírat opakovaně a existují grafy, na kterých má exponenciální časovou složitost vzhledem k n .

Nejkratší sledy v grafech bez záporných cyklů

- **Dijkstra** vyžadoval kladné délky hran.
- Argumenty o nejkratších sledech ale zůstávají v platnosti i pro grafy se zápornými délkami hran, pokud nepřipustíme záporné cykly.

Lemma (o zjednodušování sledů 2)

Uvažujme orientovaný graf se zápornými délkami hran bez záporných cyklů. Pak pro každý uv -sled existuje uv -cesta stejné nebo menší délky.

Relaxační algoritmy

- Porovnáme-li Jarníkův algoritmus s binární haldou z minulé přednášky s **DijkstraHaldou**, vidíme, že se liší **pouze** v definici klíčů prvků haldy a následně tedy i v aritmetickém výrazu pro relaxaci.
- Oba patří do třídy tzv. **relaxačních algoritmů**.
- **Dijkstra** si nyní zobecníme. Připouštíme **obecné ohodnocení** hran.
- Druhé zobecnění bude spočívat v tom, že místo otevřeného vrcholu s nejmenší hodnotou $h(v)$ vybereme **libovolný** vrchol v s **konečným** ohodnocením $h(v)$, ten relaxujeme a následně uzavřeme.
- Může se tedy stát, že uzavřený vrchol bude znovu otevřen.
- Stavy vrcholů tedy budou znamenat toto:
 - ▶ **otevřený**: jeho hodnota se změnila a je potřeba ho relaxovat,
 - ▶ **uzavřený**: byl relaxován, nevyžaduje akci.

Algoritmus Relaxace($G, \ell: E \rightarrow \mathbb{R}, v_0$)

(1) Pro všechny vrcholy v :

(2) stav(v) := nenalezený

(3) $h(v) := +\infty$; $P(v) := \perp$

(4) stav(v_0) := otevřený

(5) $h(v_0) := 0$

(6) Dokud existují otevřené vrcholy:

(7) $v := \text{nějaký otevřený vrchol}$

(8) Pro všechny následníky w vrcholu v : //relaxace v

(9) Pokud $h(w) > h(v) + \ell((v, w))$:

(10) $h(w) := h(v) + \ell((v, w))$

(11) stav(w) := otevřený

(12) $P(w) := v$

(13) stav(v) := uzavřený

(14) Vrať pole vzdáleností h a pole předchůdců P

Vlastnosti algoritmu Relaxace

Následující 3 vlastnosti platí pro jakoukoli strategii výběru otevřených vrcholů pro relaxaci

Vlastnost O: Ohodnocení $h(v)$ nikdy neroste. Je-li $h(v)$ konečné číslo, rovná se délce nějakého sledu z v_0 do v .

Vlastnost D: Pokud se výpočet zastaví, uzavřené jsou právě vrcholy dosažitelné z v_0 .

Vlastnost V: Pokud se výpočet zastaví, $h(v)$ uzavřeného vrcholu je rovno vzdálenosti $d(v_0, v)$.

Vlastnost O (ohodnocení)

Ohodnocení $h(v)$ nikdy neroste. Je-li $h(v)$ konečné číslo, rovná se délce nějakého sledu z v_0 do v .

Důkaz.

- Indukcí podle doby běhu algoritmu.
- Na počátku výpočtu tvrzení určitě platí, protože jediné konečné ohodnocení je $h(v_0) = 0$.
- Kdykoliv algoritmus snižuje ohodnocení $h(w)$, stane se tak relaxací otevřeného vrcholu v s konečným $h(v)$, jehož následníkem je w .
- Podle indukčního předpokladu tedy existuje v_0v -sled délky $h(v)$.
- Jeho rozšířením o hranu vw vznikne v_0w -sled délky $h(v) + \ell((v, w))$, což je přesně nová konečná hodnota $h(w)$. $\square$

Vlastnost D (dosažitelnost)

Pokud se výpočet zastaví, uzavřené jsou právě vrcholy dosažitelné z v_0 .

Důkaz.

- Dokážeme stejně jako obdobnou vlastnost **BFS**, s tím rozdílem, že uzavřený vrchol je možné znovu otevřít.
- Prvním otevřeným a následně uzavřeným vrcholem je v_0 .
- Vrchol je poprvé otevřen, právě když je do té doby nenalezeným následníkem dříve otevřeného vrcholu a tudíž je dosažitelný z v_0 .
- Naopak kdyby existoval nějaký dosažitelný, ale neuzavřený vrchol, existoval by ten „nejbližší“ vrchol v (co do počtu hran na nejkratší cestě).
- Pokud se **Relaxace** zastaví, znovu otevře již uzavřený vrchol jen konečně-krát, takže stačí uvážit situaci při posledním uzavření předchůdce v na nejkratší cestě. □

Vlastnost V (vzdálenost)

Pokud se výpočet zastaví, je $h(w) = d(v_0, w)$ pro všechny $w \in V$.

Důkaz.

- Inspirujeme se důkazem obdobné vlastnosti **BFS**.
- Vrchol w dosažitelný z v_0 nazveme **špatný**, pokud $h(w) \neq d(v_0, w)$.
- Z Vlastnosti O víme, že $h(w)$ odpovídá délce nějakého v_0w -sledu a tedy $h(w) > d(v_0, w)$.
- Obdobně jako pro **BFS** ukážeme sporem, že nemůže existovat žádný špatný vrchol, čímž bude důkaz dokončen.
 - ▶ Myšlenka je opět stejná – budeme předpokládat, že nějaké špatné vrcholy existují.
 - ▶ Vybereme si mezi nimi ten „nejbližší“ k v_0 a ukážeme, že nemohl být špatný.
 - ▶ Kdyby ale nějaký špatný byl, musel by být i „nejbližší“.

Vlastnost V (vzdálenost)

Pokud se výpočet zastaví, je $h(w) = d(v_0, w)$ pro všechny $w \in V$.

Pro spor předpokládejme, že existují špatné vrcholy.

- Víme, že v_0 není špatný, protože $h(v_0) = 0 = d(v_0, v_0)$.
- Buď w špatný vrchol takový, že nejkratší cesta v G z v_0 do w používá nejmenší možný počet hran.
- Buď v předchůdce w na této cestě z v_0 .
- Kdyby v byl špatný, volili bychom jej místo w , tedy platí $h(v) = d(v_0, v)$.
- Vrchol v byl jistě někdy otevřený a při jeho poslední relaxaci (tj. než byl naposledy uzavřen) muselo platit, že aktuální hodnota $h(w)$ byla větší než $d(v_0, w)$ (neboť toto platí dokonce až po zastavení Relaxace).
- V tom případě ale proběhla změna ohodnocení $h(w)$ a tedy musí platit $h(w) = d(v_0, v) + \ell((v, w)) = d(v_0, w)$. Spor. □

Bellmanův-Fordův algoritmus

- **Bellmanův-Fordův algoritmus** je relaxační algoritmus pro výpočet vzdáleností v grafech s hranami záporných délek, ale **bez záporných cyklů**.
- Namísto haldy (tj. prioritní fronty) jsou otevřené vrcholy ukládány do obyčejné fronty.
- Jinými slovy, otevřený vrchol k relaxaci zvolíme vždy jako nejstarší (tedy první) vrchol ve frontě otevřených vrcholů.
- Prvně otevřené vrcholy řadíme na konec fronty.

Algoritmus **Bellman-Ford**($G, \ell: E \rightarrow \mathbb{R}, v_0$)

(1) Pro všechny vrcholy v :

(2) stav(v) := nenalezený; $h(v) := +\infty$; $P(v) := \perp$

(3) stav(v_0) := otevřený; $h(v_0) := 0$

(4) **Vlož v_0 do fronty**

(5) **Dokud je fronta neprázdná:**

(6) **Vyjmi první vrchol z fronty a označ ho v**

(7) Pro všechny následníky w vrcholu v :

(8) Pokud $h(w) > h(v) + \ell((v, w))$:

(9) $h(w) := h(v) + \ell((v, w))$

(10) **Pokud stav(w) $\neq$ otevřený**

(11) **Přidej w do fronty**

(12) stav(w) := otevřený

(13) $P(w) := v$

(14) stav(v) := uzavřený

(15) Vrať pole vzdáleností h a pole předchůdců P

Fáze výpočtu Bellman-Ford

Definice

Definujeme **fáze výpočtu** následovně:

- Nechť $A_0 = \{v_0\}$.
- Fáze F_0 = otevření počátečního vrcholu v_0 , čili všech prvků množiny $A_0 = \{v_0\}$.
- A_i je množina vrcholů, které byly otevřeny při relaxování vrcholů z množiny A_{i-1} .
- Vytvoření množiny vrcholů A_i se nazývá fáze F_i .

Pozorování

- Pokud se změní hodnota uzavřeného vrcholu, je znovu otevřen.
- Hodnota otevřeného vrcholu se může změnit vícekrát, dříve než je vyjmut z fronty, relaxován a uzavřen.
- Během výpočtu může tedy daný vrchol postupně patřit do více A_i .

Význam $h(v)$ v Bellman-Ford

Vlastnost H

Pro každý nalezený vrchol v platí na konci fáze F_i , že $h(v) \leq \ell(S)$, kde S je nejkratší v_0v -sled o nejvýše i hranách.

Důkaz.

- Tvrzení dokážeme indukcí podle i .
- Pro $i = 0$ tvrzení platí: jediný vrchol dosažitelný z v_0 sledem s 0 hranami je v_0 sám; $h(v_0) = 0$ a pro ostatní vrcholy v je $h(v) = +\infty$.
- Dále nechť $i > 0$.
- Zvolme nějaký nalezený vrchol v na konci fáze F_i .
- Označme S nejkratší v_0v -sled o nejvýše i hranách.
- Dále označme $e(S)$ počet hran ve sledu S .
- ...

Dokončení důkazu.

- Pokud $e(S) < i$, platilo $h(v) \leq \ell(S)$ už na konci předchozí fáze F_{i-1} a jelikož ohodnocení vrcholů nerostou, platí nadále.
- Pokud $e(S) = i$, označme uv jeho poslední hranu a S' podsled z v_0 do u .
- Na konci fáze F_{i-1} je z indukce $h(u) \leq \ell(S')$.
- Na tuto hodnotu muselo být $h(u)$ nastaveno nejpozději ve fázi F_{i-1} , čímž byl vrchol u otevřen.
- Nejpozději ve fázi F_i proto musel být vrchol u relaxován.
- Na začátku relaxace muselo stále platit $h(u) \leq \ell(S')$, hodnota $h(u)$ se sice mohla změnit, ale ne zvýšit.
- Po relaxaci tedy muselo platit
$$h(v) \leq h(u) + \ell((u, v)) \leq \ell(S') + \ell((u, v)) = \ell(S).$$



Konečnost Bellman-Ford

Důsledek

Nechť $G = (V, E)$ a $n = |V|$. Pokud G neobsahuje záporné cykly, po fázi F_n se Bellman-Ford zastaví.

Důkaz.

- Po fázi F_{n-1} jsou ohodnocení všech vrcholů shora omezena délkami nejkratších cest, protože nejkratší cesty mají nejvýše $n - 1$ hran.
- Ve fázi F_n se tedy ohodnocení vrcholů již nemohou změnit a algoritmus už žádný vrchol neotevře a zastaví se. □

Složitost Bellman-Ford

Věta

V grafu bez záporných cyklů nalezne **Bellman-Ford** všechny vzdálenosti z vrcholu v_0 v čase $O(|V| \cdot |E|)$.

Důkaz.

- Podle předchozího důsledku se po $|V|$ fázích algoritmus zastaví a podle Vlastnosti D o dosažitelnosti vrcholů a Vlastnosti H o významu $h(v)$ na konci relaxačního algoritmu vydá správný výsledek.
- Během jedné fáze je přitom každý vrchol relaxován nejvýše jednou, takže celá fáze dohromady trvá $O(|E|)$. □

Jednoduchá implementace Bellman-Ford

Algoritmus **SimpleBellman-Ford**($G, \ell: E \rightarrow \mathbb{R}, v_0$)

(1) Pro všechny vrcholy v :

(2) $h(v) := +\infty; P(v) := \perp$

(3) $h(v_0) := 0$

(4) Pro $i := 1, \dots, n$:

(5) Pro každou hranu $(u, v) \in E(G)$:

(6) Pokud $h(v) > h(u) + \ell((u, v))$:

(7) $h(v) := h(u) + \ell((u, v))$

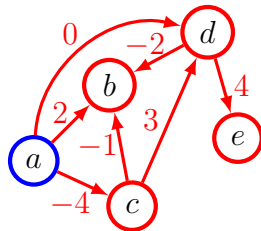
(8) $P(v) := u$

(9) Vrať pole vzdáleností h a pole předchůdců P

Algoritmus

SimpleBellman-Ford($G, \ell: E \rightarrow \mathbb{R}, v_0$)

- (1) Pro všechny vrcholy v :
- (2) $h(v) := +\infty$; $P(v) := \perp$
- (3) $h(v_0) := 0$
- (4) Pro $i := 1, \dots, n$:
- (5) Pro každou hranu $(u, v) \in E(G)$:
- (6) Pokud $h(v) > h(u) + \ell((u, v))$:
- (7) $h(v) := h(u) + \ell((u, v))$
- (8) $P(v) := u$
- (9) Vrať pole vzdáleností h
a pole předchůdců P



	a	b	c	d	e
h :	0	2	-4	3	4
P :	$\perp$	a	a	c	d

$u_i \neq 1$ $u_i \neq 2$
 $i = 3$ Pro $i > 3$
 beze změny

Hrany se procházejí v pořadí:

$(a, b), (d, e), (c, b), (a, d), (d, b), (c, d), (a, c)$.

Vlastnosti algoritmu SimpleBellman-Ford

Obdobně jako pro algoritmus Bellman-Ford bychom pro algoritmus SimpleBellman-Ford mohli dokázat:

- Ohodnocení $h(v)$ nikdy neroste.
- Je-li $h(v)$ konečné, rovná se délce nějakého sledu z v_0 do v .
- Pro každý vrchol v na konci i -té iterace cyklu na řádku (4) platí, že $h(v)$ je shora omezeno délkou nejkratšího v_0v -sledu o nejvýše i hranách.

Věta

Algoritmus SimpleBellman-Ford je korektní a pracuje v čase $O(|V| \cdot |E|)$.

Co bychom po dnešku už měli znát

- Vzdálenost v ohodnoceném grafu, nejkratší cesta.
- Relaxace vrcholu.
- Dijkstrův algoritmus.
- Bellmanův-Fordův algoritmus.